States

The `State` and the `Status` work the same as in the market actor. One is the type, and the other represents the moving parts (the variables):

```
object Bet {
  ...

  final case class Status(                                  ❶
      betId: String,
      walletId: String,
      marketId: String,
      odds: Double,
      stake: Int,
      result: Int)
      extends CborSerializable
  object Status {
    def empty(marketId: String) =                           ❷
      Status(marketId, "uninitialized", "uninitialized", -1, -1, 0)
  }
  sealed trait State extends CborSerializable {             ❸
    def status: Status
  }
  ...
}
```

❶ Variables that represent the state of the bet

❷ To be used in creating the market

❸ Placeholder for the Status, which can represent any market state

From an uninitialized state, the bet can open, settle, and close if successful. Otherwise, it fails. In either case, it can be canceled. This is reflected in the following states:

```
object Bet {
  ...

  sealed trait State extends CborSerializable {
    def status: Status
  }
```